

SYSTEMVERILOG / UVM

시각 학습 노트

구조를 보고 · 손으로 그리고 · 말로 설명하기

1단계 : 객체지향 문법

class → object → handle → inheritance → polymorphism

A4 LANDSCAPE · PRINT & ANNOTATE

왼쪽에서 개념을 읽고, 오른쪽 빈 공간에 상태와 화살표를 직접 그립니다.

객체지향 문법 전체 지도

UVM의 transaction과 component를 읽기 위한 기초

1 설계와 생성

class · object · new()

2 참조와 복사

handle · shallow/deep

3 확장

inheritance · overriding

4 동적 선택

virtual · polymorphism

5 추상화

abstract · pure virtual

6 재사용과 범위

parameter · static · 접근 제한

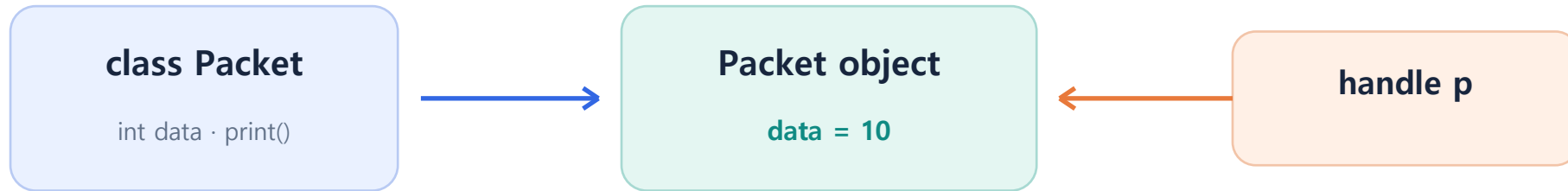
UVM 연결

transaction은 object이며 handle로 이동한다.

factory와 polymorphism은 부모 type으로 여러 자식 transaction/component를 교체 가능하게 만든다.

설계도, 실체, 그리고 handle

선언과 생성을 분리해서 생각한다



Packet p; handle만 선언 · p == null

p = new(); object 생성 · p가 object를 가리킴

손으로 상태를 그려보기

마지막 a와 b가 가리키는 object · data 값 · object 수를 표시

Packet a, b;

a = new();

a.data = 7;

b = a;

a = new();

a.data = 30;

같은 값인가, 같은 object인가

복사의 핵심은 중첩 handle 공유 여부

SHALLOW COPY



Packet 2개 · Header 1개 · b.hdr.id 변경은 a에서도 보임

DEEP COPY



Packet도 Header도 각각 독립

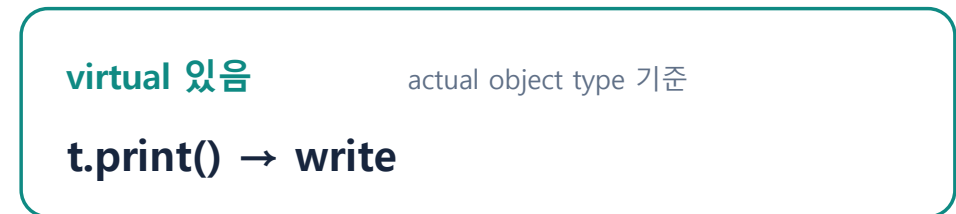
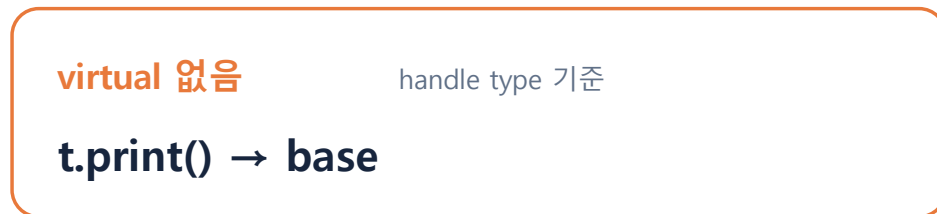
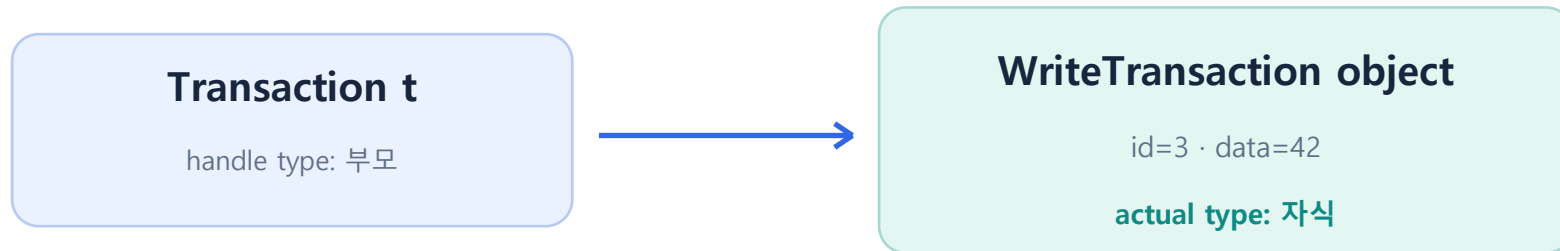
판별

b.hdr.id = a.hdr.id → int 값 복사

b.hdr = a.hdr → handle 대입 · 공유

handle type과 actual object type을 분리하라

virtual은 실제 object에 맞는 구현을 선택한다



말로 설명하기

“부모 handle 하나로 여러 자식 object의 동작을 선택할 수 있다.”

구현을 요구하고, type을 바꿔 재사용한다

ABSTRACT CLASS

virtual class Transaction

pure virtual function void print();



부모 object 생성 × 부모 handle 선언 ○

자식 print()
구현 필수

PARAMETERIZED CLASS

class Box #(type T = int);

Box#(int)

Box#(string)

서로 다른 class type · int는 기본 type

OVERRIDING

자식이 부모와 같은 method의 자기 구현을 제공한다.

부모 구현 자체는 바뀌지 않는다. SystemVerilog는 일반적인 method overloading을 지원하지 않는다.

핵심

추상 class는 공통 계약을 만들고, parameterized class는 공통 구조를 여러 type에 재사용한다.

누구의 값이며, 어디까지 보이는가

instance/static과 접근 제한자를 한 장에서 비교

INSTANCE

a.id=1 b.id=2

object마다 별도 저장

STATIC

Packet::count = 2

class가 하나를 공유 · object 없이 static method 호출

접근 가능 범위

선언	같은 class	자식 class	class 밖
int data	○	○	○
protected int data	○	○	×
local int data	○	×	×

Constructor

handle 선언 → count 변화 없음

new() 실행 → constructor 안의 count++ 실행

현재 object와 부모 구현을 구분한다

직접 그리며 1단계를 점검한다

this.data = data;

현재 object property ← argument

super.new(id);

부모 constructor 호출

그리기 문제

```
Transaction t;
WriteTransaction w;

w = new(3, 42);

t = w;
```

handle type · actual object type · object 수 · id/data를 표시

말로 답하기

① t = w가 object를 만드는가?

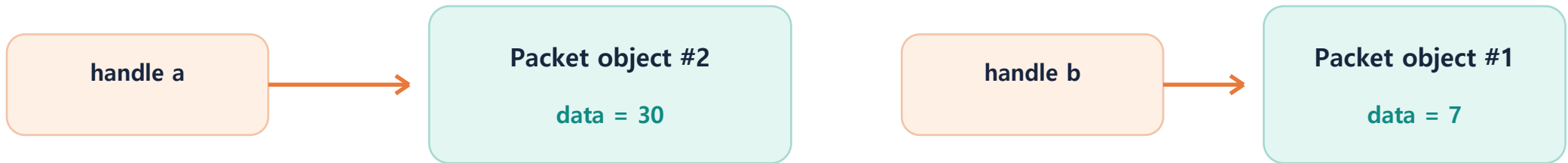
② virtual print()이면 어느 구현?

③ shallow copy의 공유 지점?

그리기 문제 답안

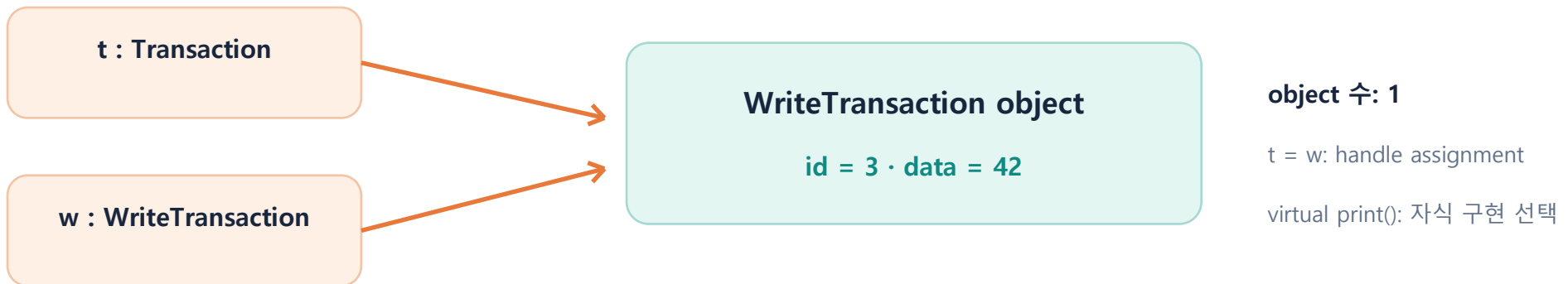
먼저 직접 그린 뒤 비교한다

3페이지 답안



object는 총 2개. b = a에서는 공유했다.
이후 a = new()가 a의 연결 대상만 바꾼다.

8페이지 답안



정답을 외우기보다 “new() 횟수 → handle 연결 → actual object type” 순서로 그린다.

데이터를 담고, process를 연결한다

자료구조 선택과 thread 동기화의 기준을 세운다

1 배열 배치
packed / unpacked

2 크기 관리
fixed / dynamic

3 가변 저장
queue / associative

4 의미 있는 type
enum / struct / typedef

5 동시 실행
fork / join 계열

6 동기화
event / semaphore / mailbox

선택 질문

크기가 고정인가? 순서인가 ID인가? 데이터를 전달하는가? 자원 접근을 제한하는가?
구조 이름을 외우기보다 해결하려는 문제부터 구분한다.

변수 이름 앞은 packed, 뒤는 unpacked

폭과 element 수를 분리해서 읽는다

logic [7:0] memory [0:15];

[7:0] · PACKED

각 element의 폭 = 8bit



[0:15] · UNPACKED

element 수 = 16개

memory[3]

→ logic [7:0] element 하나

memory[3][2]

→ 네 번째 element의 bit 2

주의

int는 element type이다. Array 크기나 element 개수와 혼동하지 않는다.

크기와 접근 방식으로 자료구조를 고른다

FIXED ARRAY

int a[4]

선언 시 크기 고정

연속 index

DYNAMIC ARRAY

int a[]

new[n]으로 재할당

연속 index

QUEUE

int q[\$]

push / pop으로 자동 증감

순서 기반

ASSOCIATIVE

int a[string]

사용한 key만 존재

ID / key 기반

UVM 연결

Queue = in-order pending 목록 · Associative array = transaction ID 기반 out-of-order matching

enum은 값의 집합, struct는 field의 묶음

typedef는 재사용할 type 이름을 만든다

ENUM

IDLE · READ · WRITE · ERROR

숫자 대신 의미 있는 상태 이름

`state.name() → "WRITE"`

STRUCT PACKED

opcode | address | data

관련 field를 하나의 value로 묶음

4 + 8 + 16 = 28bit vector

STRUCT 대입

field 값 복사 · 이후 두 variable은 독립

CLASS 대입

handle 대입 · 같은 object를 공유

구분

enum은 named value type · struct는 복합 value type · typedef는 type의 별칭

fork는 child thread를 만들고, join은 parent의 대기를 결정한다

join

모두 끝날 때까지 대기
남은 child 없음

join_any

하나가 끝나면 진행
나머지는 계속 실행

join_none

즉시 parent 진행
child는 계속 실행

wait fork

사용 지점

모든 child 종료까지 대기

join_none 뒤에 다시 합류

disable fork

사용 지점

활성 child 종료

join_any 뒤 forever 정리

합정

join_any는 남은 child를 자동으로 종료하지 않는다.

event는 알림, semaphore는 공유 자원 잠금

EVENT

-> done; trigger

@done; 다음 trigger 대기

데이터 저장 × · 과거 trigger 기억 ×

대기 전에 발생하면 event 유실 가능

SEMAPHORE

get(1) key 없으면 대기

try_get(1) 즉시 1 / 0

put(1) key 반환

put 누락 → 다른 thread deadlock

선택

발생 사실만 알림 → event

공용 bus의 동시 접근 제한 → semaphore

데이터 전달과 기다림을 한 구조에 담는다

producer와 consumer 사이의 synchronized communication



put 가득 차면 대기 **저장**

try_put 즉시 성공/실패 **저장**

get 비면 대기 **제거**

try_get 즉시 성공/실패 **제거**

peek 비면 대기 **유지**

try_peek 즉시 성공/실패 **유지**

QUEUE와 차이

queue는 저장 구조 · mailbox는 process 사이의 대기과 전달까지 제공

2단계 선택 기준과 오류 패턴

왼쪽 문제를 먼저 가리고 답한다

문제

1. sparse transaction ID 조회
2. producer → consumer 전달
3. 공용 bus 동시 접근 제한
4. reset 완료 사실만 알림
5. join_any 뒤 forever thread 정리

답

1. associative array
2. mailbox
3. semaphore
4. event
5. disable fork

Deadlock 점검

get() 대기 · put() 누락 · event 유실 · forever child · 너무 넓은 disable fork 범위

기억 순서

저장 방식 → index/key → blocking 여부 → 종료 조건

조건을 만족하는 값을 만들고, 실패를 통제한다

Randomization은 값 생성이 아니라 해를 찾는 과정이다

1 대상

rand / randc

2 조건

constraint / inline

3 집합과 분포

inside / dist

4 관계

implication / if-else

5 우선순위

solve before / soft

6 제어와 점검

mode / hook / failure

SOLVER 질문

무엇을 바꿀 수 있는가? 모든 조건의 교집합은 존재하는가? 분포는 의도와 같은가?

실패하면 이전값이 남으므로 반환값을 반드시 확인한다.

rand는 반복 가능, randc는 주기 안에서 비반복

기존값은 rand가 활성화되어 있으면 solver가 바꿀 수 있다

rand bit [1:0] id;

가능한 예 **0, 0, 2, 1**

같은 값 연속 가능

randc bit [1:0] id;

두 주기 예 **2,0,3,1 | 1,3,0,2**

주기 내부 비반복 · 경계 반복 가능

if (!item.randomize())

실패 시 0 · field는 이전값 유지

``uvm_fatal("RANDFAIL", "Item randomization failed")`

주의

실패 반환값을 무시하면 stale transaction이 driver로 전달될 수 있다.

초기값 검사가 아니라, 모든 조건을 만족하는 해 탐색

활성 rand field는 미지수이고 비활성 field는 고정값이다

RAND MODE ON

현재 addr = 12

조건 $\text{addr} \in [0:3]$

12를 0~3 중 하나로 변경 → 성공

RAND MODE OFF

고정 addr = 12

조건 $\text{addr} \in [0:3]$

12 $[0:3]$ · 변경 불가 → 실패

constraint

대입문 × randomize가 끝났을 때 반드시 참이어야 하는 식

block 내부

각 식은 기본적으로 AND · solver는 동시에 만족하는 조합을 탐색

판별 순서

바꿀 수 있는 변수 → 활성 조건 → 교집합 → 반환값

inside는 값 공간을 만들고, dist는 상대적 비중을 준다

```
addr inside {[2:5], 9, [12:14]};
```

가능값 $2 \cdot 3 \cdot 4 \cdot 5 \cdot 9 \cdot 12 \cdot 13 \cdot 14$

범위 weight 비교

[0:1] := 10

각 값에 10

[2:5] := 40

각 값에 40

[0:1] :/ 10

전체 10을 분배

[2:5] :/ 40

전체 40을 분배

기억

weight는 횟수 보장이 아니다. 짧은 실행에서 실제 비율은 달라질 수 있다.

한쪽만 제한할지, 모든 경우를 제한할지 선택한다

IMPLICATION

write -> addr inside {[8:15]};

write=1 **addr 8~15**

write=0 이 식은 addr를 제한하지 않음

CONDITIONAL

if (write) addr inside {[8:15]};

else addr inside {[0:3]};

write=1 **addr 8~15**

write=0 **addr 0~3**

논리 해석

$A \rightarrow B \equiv (!A) \text{ OR } B$

A가 거짓이면 implication은 참

선택

조건이 참일 때만 제한 → implication · 두 경우 모두 제한 → if / else

solve before는 분포, soft는 양보 가능한 기본 조건

solve a before b;

a=2 → b 3개

a=3 → b 2개

a=4 → b 1개

a를 먼저 선택: 각 a 약 1/3

soft addr inside {[0:10]};

inline: addr inside {[7:13]};

교집합 7~10 존재

soft 유지 · 최종 7~10

교집합이 없을 때 soft가 양보

공통점

합법 범위와 선택 분포를 구분해서 생각한다.

주의

solve before는 절차적 실행 순서가 아니며, soft는 다른 조건이 생겼다고 즉시 사라지지 않는다.

무엇을 랜덤화하고 어떤 조건을 적용할지 실행 중 조정한다

addr.rand_mode(0)

addr를 현재값으로 고정

관련 **constraint**는 계속 활성

c_addr.constraint_mode(0)

해당 constraint block 비활성

field의 rand mode와는 독립

pre_randomize()

mode와 상태 준비

solver 실행 전

rand field 대입값은 다시 변경 가능

post_randomize()

파생값 계산

성공 후 실행

checksum · formatted field

상속

다른 이름 → AND로 추가 같은 이름 → 자식 constraint가 부모 constraint override

3단계 실패 분석 순서

값이 이상할 때 solver 관점으로 역추적한다

1 RAND MODE

현재값을 바꿀 수 있는가?

2 ACTIVE CONSTRAINT

어떤 class/inline 조건이 켜졌는가?

3 SOLUTION SPACE

모든 조건의 교집합이 존재하는가?

4 DISTRIBUTION

dist와 solve before가 의도와 같은가?

5 RETURN VALUE

실패를 보고하고 전달을 멈췄는가?

상태 구분

conflicting = 해 없음 · over-constrained = 합법 상황을 과도하게 제외

under-constrained = 필요한 조건이 빠져 의도하지 않은 값 생성

한 문장

Randomization은 바꿀 수 있는 변수로 모든 활성 조건을 만족하는 해를 찾는 과정이다.

신호를 묶고, 역할과 시점을 분리한다

Interface는 연결 구조이고 clocking block은 timing 계약이다

1 신호 묶음

interface / instance

2 접근 역할

modport

3 class 연결

virtual interface

4 접근 시점

clocking block

5 event region

Active / NBA / Observed

6 안전한 관찰

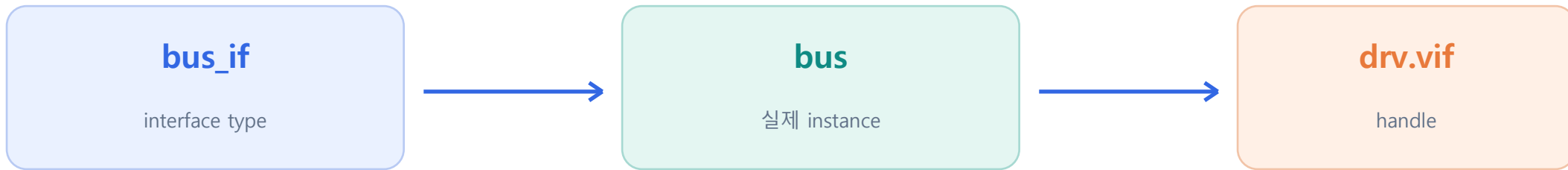
driver / monitor timing

핵심 질문

실제 신호는 어디에 있는가? 누가 읽고 쓰는가? 어느 event region에서 관찰하는가?

같은 clock edge라는 말만으로는 구동과 sampling 순서가 결정되지 않는다.

type, instance, virtual handle을 분리해서 읽는다



```
bus_if bus(.clk(clk));
```

top에서 실제 interface 생성

```
drv.vif = bus;
```

복사나 생성이 아니라 handle 연결

clk

top의 clock을 interface가 입력으로 받음

signal

interface 내부 protocol 신호

modport는 방향, clocking block은 시점

MODPORT DRV

output **valid · data**

input **ready · clk**

누가 무엇을 읽고 쓰는가?

CLOCKING drv_cb

event **@(posedge clk)**

skew **input #1step · output #0**

언제 구동하고 sampling하는가?

MON

모든 신호 input · 관찰만 수행 · interface 신호를 구동하지 않음

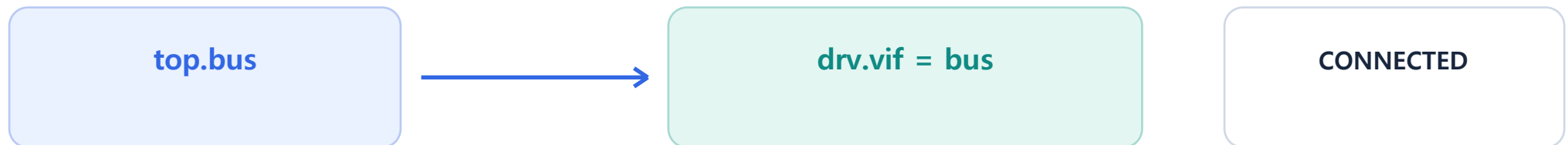
구분

방향 오류는 modport에서, timing race는 clocking block에서 막는다.

class는 실제 interface를 만들지 않고 handle로 가리킨다



정상 연결



필수 점검

if (vif == null) → fatal 처리 후 signal 접근 중단

같은 posedge라도 update와 관찰 시점은 다르다

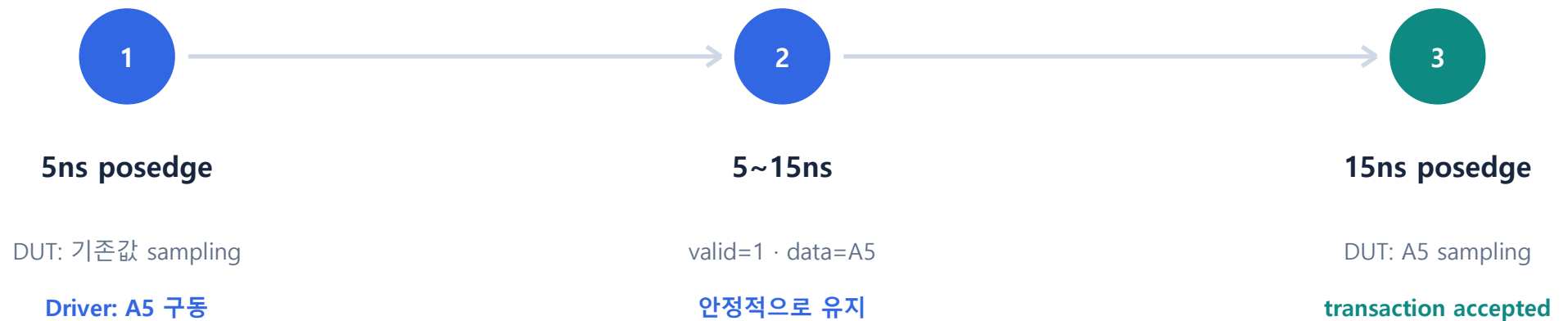


blocking = Active에서 즉시 update
위험 **DUT와 TB 실행 순서에 의존**

nonblocking <= NBA에 update 예약
효과 **Active에서는 이전값 유지**

RACE 같은 region의 실행 순서를 가정하면 simulator와 코드 변화에 따라 결과가 달라질 수 있다.

현재 edge에서 구동하고, 다음 edge에서 DUT가 sampling한다



drv_cb output #0

같은 edge에서 DUT와 먼저 실행되려고 경쟁하지 않는다.

다음 sampling edge에 사용할 값을 cycle 사이에 안정적으로 유지한다.

관점

drive edge와 DUT acceptance edge를 구분한다.

같은 edge에서 이전값과 갱신값을 구분해 sampling한다

result: 10

result: 20



EDGE 직전

10



ACTIVE

10



NBA

20



OBSERVED

20

input #1step

edge 직전

sampling result

10

input #0

NBA 반영 후

sampling result

20

선택

protocol이 요구하는 관찰 시점에 따라 #1step과 #0을 선택한다.

4단계 연결과 timing 점검 순서

signal 값보다 먼저 instance, 방향, event region을 확인한다

1 INSTANCE

실제 interface가 top에 생성됐는가?

2 VIF

class handle이 실제 instance를 가리키는가?

3 MODPORT

읽기와 쓰기 방향이 역할에 맞는가?

4 CLOCKING

기준 edge와 skew가 명확한가?

5 REGION

Active, NBA, Observed 중 어디에서 보는가?

6 CYCLE

drive와 acceptance가 어느 edge인가?

피해야 할 임시 해결

race가 사라질 때까지 임의의 #1 delay 추가

timeunit과 clock 주기가 바뀌면 관찰 위치도 달라져 문제를 다시 만든다.

한 문장

Interface는 신호를 묶고, modport는 역할을, clocking block은 timing을 정의한다.